PROJETO: TC4 FEEDBACK

Plataforma Serverless de Feedback de Aulas

Relatório Técnico: Fase 4

FIAP — PÓS TECH · ARQUITETURA E DESENVOLVIMENTO JAVA

Geovanne de Souza Paes — RM 372009

Rafael Teles Espindola — RM 370772

Victor Silva Mazzola — RM 370764

Gideão Lucas de Souza Silva — RM 374230

Luis Filipi Santos Souza — RM 372233

📹 **Vídeo de apresentação do projeto:**

[link do vídeo — a definir]

Repositório:

https://github.com/GastroHubDev/NotificaHub

Sumário

1. Introdução
2. Arquitetura da solução
3. Funções e responsabilidades
4. Modelo de dados
5. API: contrato e respostas
6. Perfis de acesso
7. Segurança e governança
8. Monitoramento
9. Decisões técnicas
10. Testes automatizados
11. CI/CD
12. Como rodar o projeto
13. Estrutura do repositório
14. Observações e limitações conhecidas

1. Introdução

Descrição do problema

Uma instituição de ensino quer ouvir os estudantes sobre a qualidade das aulas e reagir rápido quando algo dá errado. Os estudantes enviam avaliações (uma descrição e uma nota de 0 a 10); avaliações muito negativas precisam chegar imediatamente aos administradores, e todo o conjunto precisa ser consolidado em um relatório periódico.

Objetivo da solução

- **Receber avaliações** de aulas por uma API HTTP.
- **Classificar a urgência** de cada avaliação pela nota e **alertar os administradores por e-mail** quando ela for crítica.
- **Gerar um relatório semanal** com média das notas, quantidade de avaliações por dia e por urgência, e a lista de avaliações; enviá-lo por e-mail e guardá-lo no S3.
- Tudo em **arquitetura serverless na AWS**, com funções de responsabilidade única, infraestrutura como código, monitoramento e deploy automatizado.

Visão geral da stack

Camada	Tecnologia
Linguagem e framework	Java 21 + Quarkus 3.39 (`quarkus-amazon-lambda`, `quarkus-amazon-lambda-rest`)
Computação	AWS Lambda (Java 21, arm64/Graviton, SnapStart)
Entrada HTTP	Amazon API Gateway (REST) com API Key e usage plan
Dados	Amazon DynamoDB (on-demand) + DynamoDB Streams
Notificação	Amazon SNS (e-mail)
Agendamento	Amazon EventBridge Scheduler
Armazenamento de relatórios	Amazon S3
Observabilidade	CloudWatch Logs, métricas (EMF), alarmes, dashboard e AWS X-Ray
Infraestrutura como código	AWS SAM (`template.yaml`) e CloudFormation (`cicd/pipeline.yaml`)
CI/CD	AWS CodePipeline + CodeBuild + CodeConnections; GitHub Actions para testes
Região	`us-east-2` (Ohio)

2. Arquitetura da solução

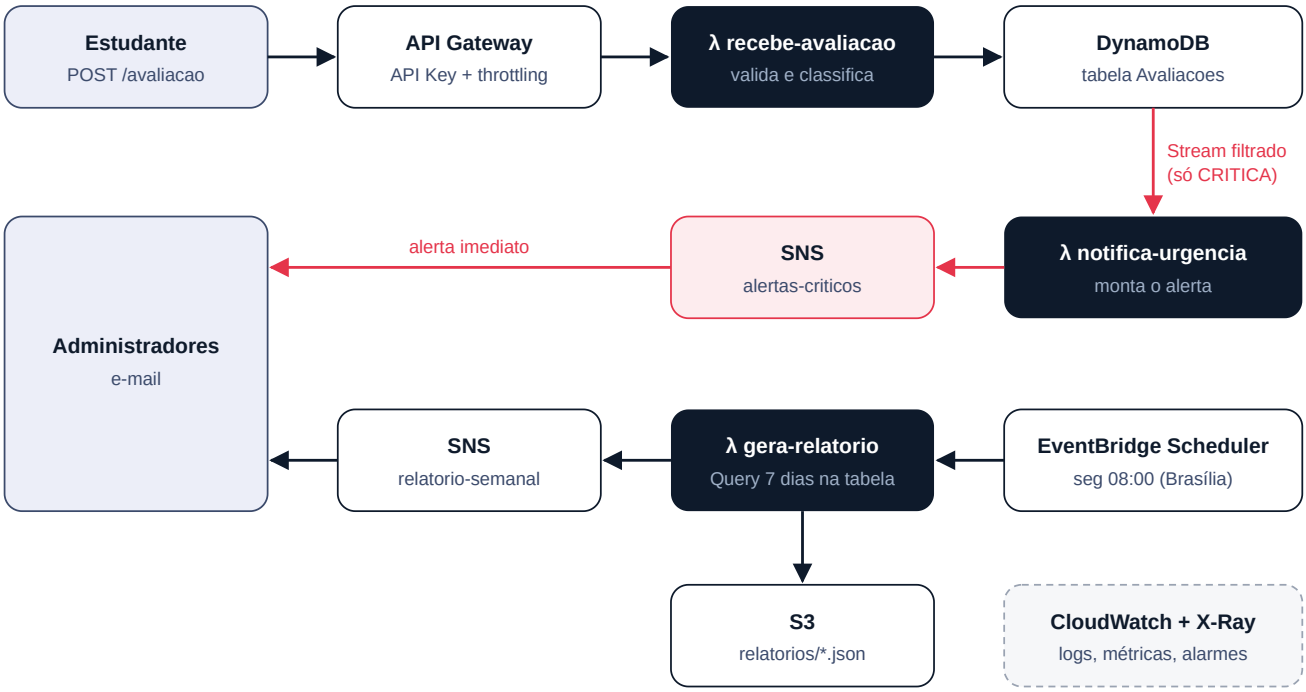


Figura 1 — Fluxos de recebimento, alerta crítico e relatório semanal. Os alarmes de erro do CloudWatch também publicam em `alertas-criticos`.

Fluxos

1. **Recebimento:** o estudante chama `POST /avaliacao` com a API Key. O API Gateway aplica throttling e encaminha para a função `recebe-avaliacao`, que valida a entrada, classifica a urgência e grava a avaliação no DynamoDB.
2. **Alerta crítico:** cada inserção gera um evento no DynamoDB Stream. Um **filtro de eventos** na integração só deixa passar inserções com `urgencia = CRITICA`; a função `notifica-urgencia` monta o alerta (descrição, urgência e data de envio) e publica no tópico SNS `alertas-criticos`, que envia o e-mail aos administradores.
3. **Relatório semanal:** toda segunda-feira às 08:00 (horário de Brasília) o EventBridge Scheduler aciona `gera-relatorio`, que consulta os 7 dias da semana anterior, calcula as estatísticas, salva o JSON no S3 e publica o resumo no tópico `relatorio-semanal`.

A função de recebimento **não sabe** que existe notificação: o desacoplamento é feito pelo Stream. Se o envio de alertas falhar, o recebimento de avaliações continua funcionando.

3. Funções e responsabilidades

Cada função tem uma única responsabilidade, um gatilho próprio e uma role IAM própria com permissão mínima.

Função	Gatilho	Responsabilidade	Permissões IAM
`tc4-recebe-avaliacao`	API Gateway `POST /avaliacao`	Validar a entrada, classificar a urgência, gravar a avaliação e emitir métricas de negócio	`dynamodb:PutItem` na tabela
`tc4-notifica-urgencia`	DynamoDB Stream (filtro `INSERT` + `CRITICA`)	Montar e enviar o alerta com descrição, urgência e data de envio	Leitura do Stream; `sns:Publish` no tópico de alertas; uso da chave KMS do tópico
`tc4-gera-relatorio`	EventBridge Scheduler `cron(0 8 ? * MON *)`, fuso `America/Sao_Paulo`	Agregar a semana: média, quantidade por dia, por urgência e lista de avaliações; salvar no S3 e enviar por e-mail	`dynamodb:Query`; `s3:PutObject` em `relatorios/*`; `sns:Publish` no tópico do relatório

Regra de urgência

A regra fica no módulo `core` (`Urgencia.classificar`) e é compartilhada pelas três funções.

Nota	Urgência	Efeito
0 – 3	**CRITICA**	Dispara alerta imediato por e-mail
4 – 6	MEDIA	Entra no relatório semanal
7 – 10	BAIXA	Entra no relatório semanal

Conteúdo do relatório semanal

- Período (segunda a domingo, horário de Brasília) e data de geração.
- Total de avaliações e **média das notas** (2 casas decimais).
- **Quantidade de avaliações por dia** — todos os 7 dias aparecem, mesmo com zero.
- **Quantidade por urgência** — CRITICA, MEDIA e BAIXA, mesmo com zero.
- Lista completa das avaliações, em ordem de envio.

O JSON completo é salvo em `s3://<bucket>/relatorios/AAAA/semana-AAAA-MM-DD.json`. O e-mail traz o resumo e até 100 avaliações (limite de tamanho do SNS), com o caminho do arquivo completo.

4. Modelo de dados

Tabela DynamoDB `Avaliacoes`, cobrança sob demanda (`PAY_PER_REQUEST`), criptografada com KMS e com Stream `NEW_IMAGE`.

Atributo	Tipo	Exemplo
`dia` (partition key)	String — dia do envio no horário de Brasília	`2026-10-27`
`criadoEm#id` (sort key)	String	`2026-10-27T14:03:11Z#9f1c…`
`id`	String (UUID)	`24ca9a9a-c9d5-…`
`descricao`	String (até 2000 caracteres)	Áudio da aula cortando
`nota`	Number (0–10)	`2`
`urgencia`	String	`CRITICA`
`dataEnvio`	String (ISO-8601, UTC)	`2026-10-29T23:09:01Z`

A chave de partição por dia faz o relatório semanal ser **7** **`Query`** (uma por dia), sem `Scan` na tabela. A sort key `criadoEm#id` mantém as avaliações ordenadas por horário dentro do dia e é única mesmo com envios no mesmo segundo.

5. API: contrato e respostas

POST `/avaliacao`

Registra a avaliação de uma aula.

Requisição

```
POST https://<api-id>.execute-api.us-east-2.amazonaws.com/prod/avaliacao
x-api-key: <chave>
Content-Type: application/json

{ "descricao": "Áudio da aula cortando", "nota": 2 }
```

Resposta 201 — criado

```
{
  "id": "24ca9a9a-c9d5-4e5f-bba0-efc06d3e43b6",
  "descricao": "Áudio da aula cortando",
  "nota": 2,
  "urgencia": "CRITICA",
  "dataEnvio": "2026-10-29T23:09:01Z"
}
```

Validações

- `descricao`: obrigatória, não vazia, até 2000 caracteres.
- `nota`: obrigatória, inteiro de 0 a 10.

Status	Quando	Exemplo de corpo
`201`	Avaliação gravada	Avaliação com `id`, `urgencia` e `dataEnvio`
`400`	Nota fora de 0–10, descrição vazia/longa ou corpo ausente	`{"title":"Constraint Violation","status":400,"violations":[{"field":"receber.request.nota","message":"nota deve estar entre 0 e 10"}]}`
`403`	Sem API Key ou chave inválida	`{"message":"Forbidden"}`
`429`	Acima do limite do usage plan	Resposta do API Gateway

Limites do usage plan `tc4-feedback-estudantes`: 10 requisições/s, rajada de 20 e cota de 10.000 requisições por dia.

6. Perfis de acesso

Perfil	Pode	Canal	Controle de acesso
Estudante	Avaliar aulas (descrição + nota 0–10)	`POST /avaliacao` no API Gateway	API Key + usage plan (throttling). Só tem acesso a esse endpoint; não lê dados
Administrador	Receber alertas de avaliações críticas	E-mail via SNS `alertas-criticos` (imediato, nota 0–3)	Inscrição no tópico (e-mail no Parameter Store)
	Receber o relatório semanal	E-mail via SNS `relatorio-semanal` (segunda 08:00)	Inscrição no tópico
	Consultar o histórico de relatórios	S3 `relatorios/AAAA/semana-*.json`	Membro do projeto AWS; bucket privado
	Acompanhar análises e a saúde do sistema	Dashboard CloudWatch `tc4-feedback`	Membro do projeto AWS

A separação é estrutural: o estudante só alcança a API de escrita. Leitura de avaliações, relatórios e métricas fica restrita a quem tem acesso ao projeto AWS ou está inscrito nos tópicos SNS.

7. Segurança e governança

- **IAM com permissão mínima:** uma role por função, cada uma restrita às ações e recursos que usa (ex.: `recebe-avaliacao` só faz `PutItem` na tabela). As trust policies exigem `aws:SourceAccount` (proteção contra *confused deputy*).
- **Acesso humano:** administradores entram como membros convidados do projeto AWS, sem usuários IAM nem chaves de acesso de longa duração.
- **API:** API Key obrigatória e usage plan com throttling e cota diária.
- **Criptografia em repouso:** DynamoDB, S3 e SNS com KMS. O tópico `alertas-criticos` usa chave própria (`alias/tc4-alertas`, com rotação automática), porque os alarmes do CloudWatch não conseguem publicar em tópico cifrado com a chave gerenciada `aws/sns`.
- **S3:** bloqueio total de acesso público, *Object Ownership* imposto, versionamento e política que nega qualquer acesso sem HTTPS.
- **Segredos fora do repositório:** o repositório é público; o e-mail dos administradores fica no Parameter Store (`/tc4-feedback/admin-email`) e é lido pelo CloudFormation no deploy.
- **CI/CD sem chaves:** o GitHub entra pela conexão CodeConnections; o CloudFormation usa uma role de deploy restrita aos recursos do projeto (prefixos `tc4-*`, tabela `Avaliacoes`, tópicos e bucket da stack).
- **Custo:** todos os serviços são cobrados por uso. O projeto roda dentro do plano gratuito da AWS.

8. Monitoramento

Recurso	O que entrega
CloudWatch Logs	Logs estruturados em JSON (campo `servico` por função), retenção de 14 dias; consultáveis no Logs Insights
Métricas de negócio (EMF)	Namespace `TC4Feedback` : `Avaliacoes` (contagem) e `Nota` , por `Urgencia` e no total. Emitidas como uma linha JSON no log, sem chamada de API nem permissão extra
AWS X-Ray	Tracing ativo no API Gateway e nas três funções
Alarmes	`tc4-recebe-avaliacao-erros` , `tc4-notifica-urgencia-erros` , `tc4-gera-relatorio-erros` : `Errors > 0` em 5 minutos → e-mail via `alertas-criticos`
Dashboard `tc4-feedback`	Avaliações por urgência (série e pizza), nota média, invocações, erros e duração p95 das funções, requisições/4XX/5XX e latência da API, estado dos alarmes

9. Decisões técnicas

9.1 Serverless com AWS Lambda

Decisão: três funções Lambda, cada uma com um gatilho e uma responsabilidade.

Por quê: a carga é intermitente (avaliações esporádicas e um relatório por semana). Lambda cobra só pelo uso, escala sozinha e não exige servidor. Funções separadas permitem permissões, alarmes e deploy independentes.

Alternativa considerada: um único serviço em container (ECS/Fargate) — custo fixo e operação maior para pouco tráfego.

9.2 Java 21 + Quarkus com SnapStart (JVM)

Decisão: Quarkus nas três funções, rodando na JVM com Lambda SnapStart, em arm64 (Graviton).

Por quê: mantém a stack Java do curso com injeção de dependência, validação (Hibernate Validator) e testes do Quarkus. O SnapStart restaura um snapshot da JVM já inicializada e reduz o cold start sem a complexidade de compilação nativa (GraalVM). Graviton custa menos por ms. As opções `-XX:TieredStopAtLevel=1` reduzem o tempo de inicialização.

Alternativa considerada: build nativo com GraalVM — cold start menor, mas build mais lento e restrições de reflexão.

9.3 DynamoDB com partição por dia

Decisão: tabela única com PK `dia` (horário de Brasília) e SK `criadoEm#id`, cobrança sob demanda.

Por quê: o único padrão de leitura é "avaliações de um dia" para o relatório; a PK por dia torna isso uma `Query` direta, sem `Scan`. O volume esperado cabe com folga no limite de uma partição. DynamoDB também oferece o Stream usado nos alertas.

Alternativa considerada: banco relacional (RDS/Aurora) — custo fixo e sem disparo nativo de eventos por inserção.

9.4 Alertas por DynamoDB Stream com filtro de eventos

Decisão: `notifica-urgencia` é acionada pelo Stream da tabela, com filtro `eventName = INSERT` e `urgencia = CRITICA`.

Por quê: desacopla recebimento e notificação — a API responde sem esperar o e-mail e continua funcionando se o SNS falhar. O filtro descarta eventos não críticos antes de invocar a função (sem custo por eles). `ReportBatchItemFailures` e `BisectBatchOnFunctionError` reprocessam só os registros que falharam, com até 3 tentativas.

Alternativa considerada: a função de recebimento publicar direto no SNS — acopla as responsabilidades e faz a API depender do envio do alerta.

9.5 SNS para e-mail

Decisão: dois tópicos SNS com inscrição por e-mail: `alertas-criticos` e `relatorio-semanal`.

Por quê: entrega e-mail sem configurar domínio nem sair do sandbox (como exigiria o SES), e permite adicionar outros administradores ou canais (SMS, fila, webhook) só criando inscrições. Os alarmes do CloudWatch reaproveitam o tópico de alertas.

9.6 EventBridge Scheduler com fuso de Brasília

Decisão: agendamento `cron(0 8 ? * MON *)` com `ScheduleExpressionTimezone: America/Sao_Paulo`, e o dia de cada avaliação calculado no mesmo fuso.

Por quê: o relatório é de negócio: "segunda às 08:00" e "semana de segunda a domingo" precisam valer no horário local, não em UTC. Uma avaliação enviada às 22:00 de domingo em Brasília (01:00 de segunda em UTC) pertence a domingo. A função também aceita `{"inicio":"AAAA-MM-DD"}` para gerar o relatório de qualquer semana sob demanda.

9.7 Relatório em JSON no S3

Decisão: cada relatório é salvo como JSON em `relatorios/AAAA/semana-AAAA-MM-DD.json`, com versionamento no bucket.

Por quê: histórico permanente e barato, legível por pessoas e por ferramentas (ex.: Athena), e sem o limite de tamanho do e-mail. O versionamento preserva relatórios regerados.

9.8 API Key + usage plan para os estudantes

Decisão: API Gateway REST com API Key obrigatória e usage plan com throttling e cota.

Por quê: o requisito é separar perfis, não identificar cada estudante. A API Key bloqueia chamadas anônimas e o usage plan protege contra abuso e custo inesperado, sem a complexidade de um provedor de identidade.

Evolução possível: Amazon Cognito para autenticação individual dos estudantes (ver seção 14).

9.9 Métricas com Embedded Metric Format

Decisão: `recebe-avaliacao` escreve as métricas de negócio como uma linha JSON no formato EMF.

Por quê: o CloudWatch extrai a métrica do log de forma assíncrona: não há chamada `PutMetricData`, latência extra nem permissão IAM adicional. Falha ao gerar a métrica nunca derruba o recebimento.

9.10 Infraestrutura como código com AWS SAM

Decisão: toda a aplicação em um único `template.yaml` (SAM); o pipeline em `cicd/pipeline.yaml` (CloudFormation), em stacks separadas.

Por quê: SAM simplifica funções, eventos (API, Stream, Scheduler), aliases e SnapStart. Separar a stack do pipeline evita que um deploy da aplicação altere a própria esteira.

9.11 CI/CD com CodePipeline em vez de GitHub Actions + OIDC

Decisão: deploy pelo AWS CodePipeline (CodeConnections → CodeBuild → CloudFormation); GitHub Actions só roda os testes.

Por quê: a política gerenciada da conta (SCP) bloqueia `iam:*Provider*`, então não é possível cadastrar o GitHub como provedor OIDC. Guardar chaves de acesso no GitHub foi descartado por segurança. Com CodeConnections nenhuma credencial sai da AWS.

9.12 Módulo `core` compartilhado

Decisão: projeto Maven multi-módulo; o domínio (`Avaliacao`, `Urgencia`, fuso de referência) fica em `core`, sem dependência de AWS.

Por quê: a regra de classificação e o cálculo do dia existem em um só lugar e são testados isoladamente; as funções dependem de interfaces (`AvaliacaoRepository`, `PublicadorAlerta`, `ArmazenamentoRelatorio`…) com implementações AWS injetadas, o que facilita os testes com mocks.

10. Testes automatizados

71 testes em 13 classes (JUnit 5, AssertJ, Mockito, REST Assured e `@QuarkusTest`), executados com `mvn verify` localmente, no GitHub Actions e no CodeBuild antes de cada deploy.

Módulo	Classes de teste	O que cobre
`core`	`UrgenciaTest`, `AvaliacaoTest`	Classificação por faixa de nota e rejeição de notas fora de 0–10; invariantes da avaliação; dia e virada do dia no horário de Brasília
`recebe-avaliacao`	`AvaliacaoResourceTest`, `DynamoDbAvaliacaoRepositoryTest`, `MetricasAvaliacaoTest`	Respostas 201/400 da API; item gravado no DynamoDB; formato EMF
`notifica-urgencia`	`NotificaUrgenciaHandlerTest`, `AvaliacaoImageTest`, `AlertaTest`	Publicação do alerta; falha parcial do lote (`batchItemFailures`); registros malformados; conteúdo do e-mail
`gera-relatorio`	`GeraRelatorioHandlerTest`, `RelatorioTest`, `SemanaTest`, `EmailRelatorioTest`, `DynamoDbAvaliacaoConsultaTest`	Semana anterior e semana informada; média, contagens por dia e urgência; limite de itens no e-mail; conversão do item da tabela

Teste de ponta a ponta em produção

Executado em 29/10/2026 no ambiente implantado:

Cenário	Resultado
`POST` com nota 2	`201`, urgência `CRITICA`, item no DynamoDB e alerta publicado no SNS
`POST` com nota 9	`201`, urgência `BAIXA`, nenhum alerta (filtro do Stream)
`POST` com nota 11	`400` — "nota deve estar entre 0 e 10"
`POST` sem API Key	`403`
Relatório da semana 26/10–01/11 sob demanda	JSON no S3 (16 avaliações, média 5,00; 9 críticas, 7 baixas) e e-mail publicado
Alarmes e logs	3 alarmes em `OK`, nenhum erro nos logs

11. CI/CD



- **Gatilho:** push na `main`. Commits que só alteram `docs/**`, `**/*.md`, `.github/**` ou `cicd/**` não disparam deploy.
- **Execução em fila** (`QUEUED`): pushes seguidos não se atropelam.
- **Testes bloqueiam o deploy:** se `mvn verify` falhar, o CloudFormation não roda.
- **Artefatos:** bucket S3 privado e cifrado, com expiração em 30 dias e cache do Maven.
- **GitHub Actions** (`.github/workflows/ci.yml`): roda os testes em todo push e pull request, sem nenhuma credencial AWS.
- **Deploy seguro das funções:** cada deploy publica uma nova versão e move o alias `live`; a API e os gatilhos sempre chamam o alias.

12. Como rodar o projeto

Pré-requisitos

- Java 21 e Maven 3.9+
- AWS CLI v2 e AWS SAM CLI
- Acesso ao projeto AWS na região `us-east-2`, com o perfil `tc4` configurado

1. Build e testes locais

```
git clone https://github.com/GastroHubDev/NotificaHub.git
cd NotificaHub
mvn verify          # roda os testes e gera */target/function.zip
```

2. Configurar o e-mail dos administradores (uma vez)

```
aws login --profile tc4
aws ssm put-parameter --name /tc4-feedback/admin-email --type String \
  --value <email> --overwrite --profile tc4 --region us-east-2
```

3a. Deploy manual

```
mvn -DskipTests package
sam deploy          # stack tc4-feedback, us-east-2 (configurado em samconfig.toml)
```

3b. Deploy automático (CI/CD)

```
aws cloudformation deploy --stack-name tc4-feedback-cicd \
  --template-file cicd/pipeline.yaml --capabilities CAPABILITY_IAM \
  --profile tc4 --region us-east-2
```

Depois, autorize a conexão uma única vez no console: **Developer Tools > Settings > Connections > notificahub-github > Update pending connection**. A partir daí, todo push na `main` faz o deploy.

> **Confirmação dos e-mails:** no primeiro deploy a AWS envia um e-mail de confirmação para cada tópico (`alertas-criticos` e `relatorio-semanal`). As mensagens só chegam depois de clicar em **Confirm subscription**.

4. Enviar uma avaliação

```
URL=$(aws cloudformation describe-stacks --stack-name tc4-feedback --profile tc4 \
  --query "Stacks[0].Outputs[?OutputKey=='ApiUrl'].OutputValue" --output text)
KEY_ID=$(aws cloudformation describe-stacks --stack-name tc4-feedback --profile tc4 \
  --query "Stacks[0].Outputs[?OutputKey=='ApiKeyId'].OutputValue" --output text)
KEY=$(aws apigateway get-api-key --api-key "$KEY_ID" --include-value \
  --profile tc4 --query value --output text)

curl -i -X POST "$URL" -H "x-api-key: $KEY" -H 'Content-Type: application/json' \
  -d '{"descricao":"Áudio da aula cortando","nota":2}'
```

Com nota de 0 a 3, o alerta chega por e-mail em alguns segundos.

5. Gerar o relatório sem esperar a segunda-feira

```
# semana anterior (comportamento do agendamento)
aws lambda invoke --function-name tc4-gera-relatorio --payload '{}' \
  --cli-binary-format raw-in-base64-out --profile tc4 --region us-east-2 saida.json

# uma semana específica (a data deve ser uma segunda-feira)
aws lambda invoke --function-name tc4-gera-relatorio --payload '{"inicio":"2026-10-26"}' \
  --cli-binary-format raw-in-base64-out --profile tc4 --region us-east-2 saida.json
```

6. Acompanhar

- Dashboard: output `DashboardUrl` da stack (CloudWatch > Dashboards > `tc4-feedback`).
- Relatórios: output `BucketRelatorios` , prefixo `relatorios/` .
- Logs: grupos `/aws/lambda/tc4-*` no CloudWatch Logs.

7. Remover os recursos

```
sam delete --stack-name tc4-feedback --profile tc4 --region us-east-2
aws cloudformation delete-stack --stack-name tc4-feedback-cicd --profile tc4 --region us-east-2
```

Os buckets S3 precisam ser esvaziados (incluindo versões) antes da remoção.

13. Estrutura do repositório

```
NotificaHub/
├── pom.xml                      # parent Maven (módulos + versões)
├── core/                        # domínio: Avaliacao, Urgencia, regra de classificação
├── recebe-avaliacao/            # λ HTTP  (POST /avaliacao)
├── notifica-urgencia/           # λ DynamoDB Stream
├── gera-relatorio/              # λ agendada (relatório semanal)
├── template.yaml                # infra SAM: tabela, SNS, S3, KMS, API, scheduler, alarmes, dashboard
├── samconfig.toml               # parâmetros do sam deploy
├── buildspec.yml                # CodeBuild
├── cicd/pipeline.yaml           # CodePipeline + CodeBuild + roles (stack tc4-feedback-cicd)
├── .github/workflows/ci.yml     # testes em push/PR
├── docs/
│   ├── ARQUITETURA.md
│   ├── RelatorioTecnicoTC4Fase4.html / .pdf
│   └── GITHUB_VIDEO_ALUNOS.txt
└── README.md
```

14. Observações e limitações conhecidas

- **Estudantes não são identificados individualmente:** a API Key é compartilhada pelo perfil. Para saber quem avaliou ou limitar uma avaliação por aluno/aula, o próximo passo seria autenticação com Amazon Cognito no API Gateway.
- **Relatórios são consultados no console AWS:** não há interface própria nem endpoint de leitura. Uma evolução seria `GET /relatorios` restrito a administradores.
- **E-mail via SNS:** o texto é simples (sem HTML) e cada destinatário precisa confirmar a inscrição. O e-mail do relatório lista até 100 avaliações; o restante fica no JSON do S3.
- **Região única:** toda a solução roda em `us-east-2`, sem replicação entre regiões.
- **Relatório regerado sobrescreve o arquivo da semana:** a versão anterior continua disponível pelo versionamento do bucket.
- **Dados de teste:** as avaliações do teste de ponta a ponta têm o prefixo `[TESTE E2E]` na descrição e foram mantidas como evidência.